1

Automatic Zoom

ADITYA MALLICKBhubaneswar, India

#beingaditya198@gmail.com +91-9348370213 §github.com/Ui-Adiii
ïlinkedin.com/in/aditya-mallick-

aditya-mallick.is-a.dev

SUMMARY

Aspiring Full Stack Developer and CSIT undergraduate with practical experience in designing and deploying scalable web ap-

plications using the MERN stack. Proficient in frontend and backend development, API integration, and database management.

Strong problem-solving mindset with the ability to collaborate effectively and leverage modern technologies to deliver impactful

solutions.

EDUCATION

• C.V. Raman Global University 2022 – 2026

B.Tech in Computer Science and Information Technology Bhubaneswar, Odisha

• Regional Higher Secondary School 2020 – 2022

Higher Secondary in Science Nimapara, Odisha

• Binayakdev High School 2020

Matriculation Puri, Odisha

SKILLS

• Programming Languages: Java

• Frontend Development: HTML, CSS, JavaScript, React.js, Next.js, TypeScript

• Backend Development: Node.js, Express.js

• Databases: MySQL, MongoDB

• Developer Tools: Git, GitHub, VS Code, Postman, Docker

EXPERIENCE

• Full Stack Developer Intern, NexaChain AI December 2025 – March 2026, Kolkata, West BengalContributed to developing web applications using modern full-stack technologies. Worked on the AURAVEST project, imple-

menting responsive interfaces, backend functionality, REST APIs, database operations, and authentication features. Collabo-

rated with mentors and team members to build scalable and maintainable software solutions.

• Full Stack Developer Intern, MindBrain Innovations Pvt. Ltd. June 2024, Bhubaneswar, OdishaContributed to building web applications with the MERN stack. Designed responsive interfaces, implemented server-side

functionality, integrated REST APIs, and handled database operations. Collaborated with mentors and teammates to solve

practical challenges, gaining hands-on experience with scalable software development.

PROJECTS

CLI Build 2026

• Built an AI-powered CLI using Bun and TypeScript with Agent, Plan, Ask, and Telegram modes for developer assistance.

• Integrated NVIDIA models with tool calling for project inspection, code assistance, and task execution.

• Implemented codebase research, step-by-step planning, Firecrawl web search, and approval-based agent operations.

Technologies: Bun, TypeScript, NVIDIA, AI SDK, Firecrawl, Telegram Bot API, Node.js

E-Commerce Platform 2024

• Developed a full-stack e-commerce platform using the MERN stack with product browsing, shopping cart, checkout, and order

management.

• Implemented JWT authentication, role-based access, product management, and secure user/admin workflows.

• Built a responsive admin dashboard with analytics and integrated Cloudinary for efficient product image management.

Technologies: MongoDB, Express.js, React.js, Node.js, Tailwind CSS, JWT, Cloudinary

Employee Management System 2025

• Built a full-stack employee and task management application using Next.js, React, Redux Toolkit, and MongoDB.

• Implemented JWT authentication, role-based access control, persistent sessions, and admin/employee dashboards.

• Designed a responsive interface with Tailwind CSS and integrated task management, analytics, and Cloudinary.

Technologies: Next.js, React, Redux Toolkit, Node.js, MongoDB, Tailwind CSS, JWT, Cloudinary

CERTIFICATES

• HTML, CSS, JS

• The Complete MySQL Bootcamp

• Problem Solving in C

• Linux Command

• Java Programming Beginners

Blog Aditya's portfolio includes personal technical notes

covering things he has learned, experimented with, and

explored while building software.

###

Why I'm Learning AI Coding Tools Description: I've been exploring AI coding tools to understand how they fit into my

development workflow, where they save time, and where I still need to figure things out myself. Slug:

why-im-learning-ai-coding-tools Tags: AI, AI Agents, Claude Code, OpenCode, Codex Date: 2026-09-02 Content:

Why I'm Learning AI Coding Tools I've been spending more time exploring AI coding tools lately. At first, I was mainly

curious about how they could help with everyday development, but the more I experimented with them, the more I

wanted to understand how to use them properly. While building projects, I sometimes get stuck on an unfamiliar

error, spend too much time figuring out an API, or go back and forth between documentation and examples. AI

coding tools can help with these situations, but I've also learned that getting an answer quickly doesn't necessarily

mean getting the right answer.

##

The Tools I'm Exploring I've been trying different tools, including Claude, OpenCode, and Codex, to see how they fit

into my workflow. I use them to explore implementation ideas, understand unfamiliar code, and think through different

approaches to a problem. They can be especially helpful when I need a starting point or want to understand

something I haven't worked with before. But the results aren't always perfect. Sometimes the suggested code looks

correct at first and then fails when I run it. Other times, the solution works but isn't the best fit for the project. That's

why I don't think it's enough to judge a tool by how impressive its responses look.

##

Getting More Out of the Tools One thing I'm learning is that the way I approach a problem matters. Instead of asking

for an entire feature immediately, I can break the task into smaller parts, provide the relevant code, and explain what

I'm trying to achieve. This makes it easier to evaluate the response and understand the reasoning behind a

suggested solution. I also try to check the documentation and test the code rather than assuming the generated

answer is correct. These tools can speed up parts of development, but I still need to understand the code I'm adding

to a project.

##

Not Every Task Needs AI I'm also realizing that using AI for everything doesn't necessarily make development faster.

For a simple problem, writing the code myself might be quicker than explaining the task, reviewing the response, and

fixing unnecessary changes. For an unfamiliar library or a complicated implementation, getting another perspective

can be much more useful. I'm trying to understand where these tools actually help instead of using them just because

they're available.

##

Learning Without Losing the Fundamentals I don't want to become dependent on a tool to explain every error or write

every function. I still want to improve at debugging, reading documentation, and understanding why a particular

solution works. For me, AI coding tools are most useful when they help me learn something or move past a problem

without hiding the details. I want to be able to review the generated code, recognize when something is wrong, and

make changes without needing the tool to do everything for me.

##

What I'm Taking Away So Far I'm still experimenting with different tools and figuring out which ones work best for

different situations. Some tasks become easier, while others still require the same amount of debugging and careful

thinking. I think that's part of learning how to use a new tool effectively. My goal isn't just to write code faster. It's to

become a better developer while learning how these tools can fit into the way I build projects. I'll keep experimenting

as I work on more projects and see what actually makes a difference.

###

Building My Own AI-Powered CLI Description: Building CLI Build helped me understand AI agents beyond simple API

calls, from tool calling and planning to controlling how an agent interacts with a codebase. Slug:

building-my-own-ai-powered-cli Tags: AI, Ai-Sdk, Bun, TypeScript, NVIDIA Date: 2026-10-5 Content:

Building My Own AI-Powered CLI I've been experimenting with different AI tools for a while, and at some point, I

wanted to understand how they actually work behind the scenes. Instead of just using them for coding, I decided to

build my own command-line tool. That's how CLI Build started. It's built with Bun and TypeScript, and the idea is to

bring AI assistance directly into the terminal for coding tasks, codebase exploration, planning, and web research.

##

Starting with Different Modes I wanted the tool to handle different kinds of requests without following the same

workflow every time. So I built three modes: Ask, Plan, and Agent. Ask mode is for getting explanations and answers.

Plan mode helps break a task into smaller steps before implementation, while Agent mode can use tools to explore a

codebase and work through tasks. Building these modes made me think more carefully about how an AI tool should

handle a request. Sometimes a direct answer is enough. Other times, it makes more sense to understand the project

first or plan the changes before doing anything.

##

Working with AI Models and Tool Calling For the AI functionality, I integrated NVIDIA models using the AI SDK. One

of the interesting parts was implementing tool calling so the model could request actions and use their results in the

next steps. I also experimented with different models and noticed that response times can vary quite a bit. This

matters more in an agent workflow because a single task can involve multiple model calls rather than just one

response. I ran into some compatibility issues along the way, especially while working with provider configurations

and structured outputs. These were useful learning experiences because they showed me that integrating an LLM

into an application involves more than just sending a request and displaying the response.

##

Letting the Agent Work with a Codebase One of the main goals was to make the tool useful for actual development

work. I worked on codebase exploration, tool-based execution, and integrating Firecrawl for web research. This gives

the agent a way to work with information beyond the original prompt. Instead of relying only on what the model

already knows, it can use available tools to gather relevant context. Getting this workflow right also means handling

tool results, errors, and the next model response properly. There are quite a few moving parts involved, and that

made this one of the more interesting areas to work on.

##

Why Agent Permissions Matter Giving an AI agent access to tools is useful, but it also introduces risks. I didn't want

the tool to execute every action without any limits or review. I added features such as approval-based execution,

staged file changes, restricted tool access, and configurable step limits. These features help keep the execution

process controlled. For example, staged changes make it possible to review modifications, while step limits prevent

an agent from continuing indefinitely. Working on this part changed how I think about AI agents. Making an agent

capable of doing more is one challenge, but deciding what it should be allowed to do is just as important.

##

Adding Telegram Integration I also integrated the Telegram Bot API to support remote interaction. This was an

interesting addition because it meant thinking about how the tool could be used outside the terminal. A command-line

interface and a messaging interface work differently, so connecting them also meant considering how users would

interact with the underlying functionality through each interface.

##

What I'm Learning from This Project This project has taught me a lot about building AI-powered applications. I've

learned more about tool calling, agent workflows, planning, model integration, and handling permissions around tool

execution. I've also realized that the model itself is only one part of the system. The surrounding application logic, tool

design, error handling, and execution limits all affect how useful the final result is. CLI Build is still something I want to

keep improving. There are more things I'd like to explore around reliability, execution workflows, and making the tool

more practical for everyday development. For now, building it has been a good way to move beyond simply using AI

coding tools and start understanding how to build one myself. You can check out the project here:

https://github.com/Ui-Adiii/cli-build

MISSING DATA Some sections may contain limited or

unavailable information. If a section is empty or does not

contain the requested information, do not infer or generate

missing details. Follow the system prompt's UNKNOWN

INFORMATION rule.

Contact Information The following contact information is

publicly available and may be shared when visitors ask for aditya's contact details.

PORTFOLIO INFORMATION — ADITYA MALLICK ## Profile Aditya Mallick is a full-stack developer focused on building modern, responsive, and user-friendly web applications. --- ## Projects Aditya has built 7 projects. ### CLI Build Description: An AI-powered developer CLI built with Bun and TypeScript for code assistance, planning, web research, and Telegram integration. Technologies: Bun, TypeScript, NVIDIA AI, AI SDK, Firecrawl, Telegram Bot API, Node.js Domains: AI, CLI, Bun, TypeScript, Developer Tools ### Full-Stack E-Commerce Platform Description: A comprehensive modern e-commerce platform built with the MERN stack, designed to provide a seamless shopping experience and robust management tools for administrators. Technologies: React, Node.js, Express, MongoDB, Tailwind CSS, Flowbite Domains: E-Commerce, MERN, Full-Stack, Shopping ### Full-Stack Blog App Description: A modern full-stack blogging platform built with MongoDB, Express.js, React.js, and Node.js featuring secure authentication, rich post creation, image uploads, comments, and user/admin dashboards. Technologies: React, Node.js, Express, MongoDB, JWT, OAuth, Tailwind CSS, Shadcn UI Domains: Blog, MERN, Full-Stack, Content Management ### Backend Ledger API Description: A secure Node.js, Express, and MongoDB backend for authentication, account management, and idempotent ledger-based money transfers. Technologies: Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, Nodemailer Domains: Backend, Node.js, Express, MongoDB, FinTech ### Authentication Backend Description: A secure and scalable Node.js authentication backend with JWT authentication, email verification, role-based access control, and multi-device session management. Technologies: Node.js, Express.js, MongoDB, Mongoose, JWT, bcryptjs, Nodemailer, Multer, Cloudinary Domains: Authentication, Node.js, Express, MongoDB ### Employee Management System (EMS) Description: A full-stack Employee Management System built with Next.js, React, Redux Toolkit, and MongoDB featuring secure authentication, role-based access, employee management, and task tracking. Technologies: Next.js, React, Redux Toolkit, MongoDB, Mongoose, Tailwind CSS, React Toastify, JWT Domains: Management, Next.js, Full-Stack, Productivity ### Full-Stack Todo App Description: A modern full-stack Todo Application built with Next.js 15, React 19, and MongoDB, featuring secure authentication, real-time task management, progress tracking, and a responsive UI. Technologies: Next.js, React, MongoDB, JWT, Tailwind CSS Domains: Productivity, Next.js, Full-Stack, MongoDB --- ## Technical Skills The following technologies are represented across Aditya's portfolio projects: - Bun - TypeScript - NVIDIA AI - AI SDK - Firecrawl - Telegram Bot API - Node.js - React - Express - MongoDB - Tailwind CSS - Flowbite - JWT - OAuth - Shadcn UI - Express.js - Mongoose - bcrypt - Nodemailer - bcryptjs - Multer - Cloudinary - Next.js - Redux Toolkit - React Toastify --- ## Project Domains Aditya's projects cover the following areas: - AI - CLI - Bun - TypeScript - Developer Tools - E-Commerce - MERN - Full-Stack - Shopping - Blog - Content Management - Backend - Node.js - Express - MongoDB - FinTech - Authentication - Management - Next.js - Productivity --- ## Professional Experience ### FullStack Intern — Nexachain.ai Period: Dec 2025 - March 2026 Project: AURAVEST — Smart Investment

Platform Type: Investment Platform - Intern Description: Worked on the development of AURAVEST, a smart investment platform, contributing to both frontend and backend development. Responsibilities: - Built and refined UI pages for the AURAVEST investment platform. - Developed and tested backend APIs to support core application functionality. - Integrated frontend components with backend services and APIs. - Worked on responsive and user-friendly interfaces across key platform pages. Technologies: React, TypeScript, Tailwind CSS, MongoDB, Express.js ### FullStack Intern — Mindbrain Innovations Pvt. Ltd. Period: June 2024 - July 2024 Project: E-Commerce — Full-Stack Shopping Platform Type: E-Commerce Platform - Intern Description: Worked on a full-stack e-commerce platform with customer shopping functionality and an administrative management system. Responsibilities: - Built and refined UI pages for the e-commerce platform. - Developed and tested backend APIs for core application functionality. - Implemented secure JWT-based authentication and protected routes. - Integrated product management with image uploads using Cloudinary. - Implemented cart, checkout, order placement, and order tracking features. - Developed admin functionality for managing users, products, and orders. - Added product search, category filtering, and responsive UI experiences. Technologies: React, Tailwind CSS, Node.js, MongoDB, Express.js, JWT, Cloudinary --- --- ## Blog Aditya's portfolio includes personal technical notes covering things he has learned, experimented with, and explored while building software. ### Why I'm Learning AI Coding Tools Description: I've been exploring AI coding tools to understand how they fit into my development workflow, where they save time, and where I still need to figure things out myself. Slug: why-im-learning-ai-coding-tools Tags: AI, AI Agents, Claude Code, OpenCode, Codex Date: 2026-09-02 Content: Why I'm Learning AI Coding Tools I've been spending more time exploring AI coding tools lately. At first, I was mainly curious about how they could help with everyday development, but the more I experimented with them, the more I wanted to understand how to use them properly. While building projects, I sometimes get stuck on an unfamiliar error, spend too much time figuring out an API, or go back and forth between documentation and examples. AI coding tools can help with these situations, but I've also learned that getting an answer quickly doesn't necessarily mean getting the right answer. ## The Tools I'm Exploring I've been trying different tools, including Claude, OpenCode, and Codex, to see how they fit into my workflow. I use them to explore implementation ideas, understand unfamiliar code, and think through different approaches to a problem. They can be especially helpful when I need a starting point or want to understand something I haven't worked with before. But the results aren't always perfect. Sometimes the suggested code looks correct at first and then fails when I run it. Other times, the solution works but isn't the best fit for the project. That's why I don't think it's enough to judge a tool by how impressive its responses look. ## Getting More Out of the Tools One thing I'm learning is that the way I approach a problem matters. Instead of asking for an entire feature immediately, I can break the task into smaller parts, provide the relevant code, and explain what I'm trying to achieve. This makes it easier to evaluate the response and understand the reasoning behind a suggested solution. I also try to check the documentation and test the code rather than

assuming the generated answer is correct. These tools can speed up parts of development, but I still need to understand the code I'm adding to a project. ## Not Every Task Needs AI I'm also realizing that using AI for everything doesn't necessarily make development faster. For a simple problem, writing the code myself might be quicker than explaining the task, reviewing the response, and fixing unnecessary changes. For an unfamiliar library or a complicated implementation, getting another perspective can be much more useful. I'm trying to understand where these tools actually help instead of using them just because they're available. ## Learning Without Losing the Fundamentals I don't want to become dependent on a tool to explain every error or write every function. I still want to improve at debugging, reading documentation, and understanding why a particular solution works. For me, AI coding tools are most useful when they help me learn something or move past a problem without hiding the details. I want to be able to review the generated code, recognize when something is wrong, and make changes without needing the tool to do everything for me. ## What I'm Taking Away So Far I'm still experimenting with different tools and figuring out which ones work best for different situations. Some tasks become easier, while others still require the same amount of debugging and careful thinking. I think that's part of learning how to use a new tool effectively. My goal isn't just to write code faster. It's to become a better developer while learning how these tools can fit into the way I build projects. I'll keep experimenting as I work on more projects and see what actually makes a difference. ### Building My Own AI-Powered CLI Description: Building CLI Build helped me understand AI agents beyond simple API calls, from tool calling and planning to controlling how an agent interacts with a codebase. Slug: building-my-own-ai-powered-cli Tags: AI, Ai-Sdk, Bun, TypeScript, NVIDIA Date: 2026-10-5 Content: Building My Own AI-Powered CLI I've been experimenting with different AI tools for a while, and at some point, I wanted to understand how they actually work behind the scenes. Instead of just using them for coding, I decided to build my own command-line tool. That's how CLI Build started. It's built with Bun and TypeScript, and the idea is to bring AI assistance directly into the terminal for coding tasks, codebase exploration, planning, and web research. ## Starting with Different Modes I wanted the tool to handle different kinds of requests without following the same workflow every time. So I built three modes: Ask, Plan, and Agent. Ask mode is for getting explanations and answers. Plan mode helps break a task into smaller steps before implementation, while Agent mode can use tools to explore a codebase and work through tasks. Building these modes made me think more carefully about how an AI tool should handle a request. Sometimes a direct answer is enough. Other times, it makes more sense to understand the project first or plan the changes before doing anything. ## Working with AI Models and Tool Calling For the AI functionality, I integrated NVIDIA models using the AI SDK. One of the interesting parts was implementing tool calling so the model could request actions and use their results in the next steps. I also experimented with different models and noticed that response times can vary quite a bit. This matters more in an agent workflow because a single task can involve multiple model calls rather than just one response. I ran into some compatibility issues along the way, especially while working with provider

configurations and structured outputs. These were useful learning experiences because they showed me that integrating an LLM into an application involves more than just sending a request and displaying the response. ## Letting the Agent Work with a Codebase One of the main goals was to make the tool useful for actual development work. I worked on codebase exploration, tool-based execution, and integrating Firecrawl for web research. This gives the agent a way to work with information beyond the original prompt. Instead of relying only on what the model already knows, it can use available tools to gather relevant context. Getting this workflow right also means handling tool results, errors, and the next model response properly. There are quite a few moving parts involved, and that made this one of the more interesting areas to work on. ## Why Agent Permissions Matter Giving an AI agent access to tools is useful, but it also introduces risks. I didn't want the tool to execute every action without any limits or review. I added features such as approval-based execution, staged file changes, restricted tool access, and configurable step limits. These features help keep the execution process controlled. For example, staged changes make it possible to review modifications, while step limits prevent an agent from continuing indefinitely. Working on this part changed how I think about AI agents. Making an agent capable of doing more is one challenge, but deciding what it should be allowed to do is just as important. ## Adding Telegram Integration I also integrated the Telegram Bot API to support remote interaction. This was an interesting addition because it meant thinking about how the tool could be used outside the terminal. A command-line interface and a messaging interface work differently, so connecting them also meant considering how users would interact with the underlying functionality through each interface. ## What I'm Learning from This Project This project has taught me a lot about building AI-powered applications. I've learned more about tool calling, agent workflows, planning, model integration, and handling permissions around tool execution. I've also realized that the model itself is only one part of the system. The surrounding application logic, tool design, error handling, and execution limits all affect how useful the final result is. CLI Build is still something I want to keep improving. There are more things I'd like to explore around reliability, execution workflows, and making the tool more practical for everyday development. For now, building it has been a good way to move beyond simply using AI coding tools and start understanding how to build one myself. You can check out the project here: https://github.com/Ui-Adiii/cli-build --- --- ## MISSING DATA Some sections may contain limited or unavailable information. If a section is empty or does not contain the requested information, do not infer or generate missing details. Follow the system prompt's UNKNOWN INFORMATION rule. --- --- ## Contact Information The following contact information is publicly available and may be shared when visitors ask for Aditya's contact details.